



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

COMPUTAÇÃO DE ALTO DESEMPENHO

DOCENTE: CARLA DOS SANTOS SANTANA

E SAMUEL XAVIER DE SOUZA

DISCENTE: CLARA VIRGÍNIA NASCIMENTO SANTOS (20240002212)

Tarefa 6

1. ENUNCIADO

Implemente em C a estimativa estocástica de π . Paralelize com `#pragma omp parallel for` e explique o resultado incorreto. Corrija a condição de corrida utilizando o `#pragma omp critical` e reestruturando com `#pragma omp parallel` seguido de `#pragma omp for` e aplicando as cláusulas `private`, `firstprivate`, `lastprivate` e `shared`. Teste diferentes combinações e explique como cada cláusula afeta o comportamento do programa. Comente também como a cláusula `default(none)` pode ajudar a tornar o escopo mais claro em programas complexos.

2. DISCUSSÃO

2.1. Método Estocástico

A estimativa estocástica de π é um método que utiliza a geração de números aleatórios para estimar o valor de π . Pontos são gerados aleatoriamente dentro de um quadrado com um círculo inscrito; a proporção de pontos que caem dentro do círculo em relação ao total de pontos gerados é usada para estimar o valor de π convergindo para esta relação sendo $\pi/4$.

2.2. Race Condition

Ao paralelizar tarefas, um problema comum é a condição de corrida, quando múltiplas threads tentam acessar e modificar simultaneamente variáveis dependentes, o que pode trazer erros de registro, com valores errôneos que prejudicam o resultado final da tarefa. Para isso, existem diversas soluções para resolver essa problemática, como:

- `#pragma omp critical`

Essa diretiva cria uma área dentro da região paralela que permite que apenas uma thread entre e execute o que está dentro do bloco critical, garantindo que a variável compartilhada seja modificada por apenas uma thread. Contudo, mesmo resolvendo o problema, essa diretiva não é a solução mais eficiente, já que as threads ficam esperando para acessar a região crítica, o que pode levar a um desempenho ruim.

- #pragma omp parallel + #pragma omp for

Essas abordagem consegue separar a execução paralela da criação das tarefas em blocos iterativos, em que cada thread é responsável por um subconjunto dos pontos a serem gerados. Isso reduz a necessidade de sincronização já que não há dependência entre as iterações, pois cada thread trabalha com seus próprios dados locais. Após isso, cada thread fica esperando ao final para fazer a sincronização dos resultados (barreira implícita).

2.3. Cláusulas de escopo na OpenMP

Essas cláusulas tem o objetivo de definir o comportamento das variáveis dentro de uma região paralela, evitando o risco de condições de corrida e garantindo que cada thread tenha acesso às variáveis corretas.

- Shared

Todas as variáveis acessam a mesma variável compartilhada, o que pode levar a condições de corrida se houver escrita simultânea.

- Private

Cada thread tem sua própria cópia da variável (com lixo de memória como valor inicialmente), evitando conflitos de acesso, mas não compartilhando os resultados entre as threads.

- Firstprivate

Semelhante a private, mas inicializa a cópia da variável com o valor da variável original antes da região paralela.

- Lastprivate

Garante que a última thread a modificar a variável atualize a variável original após a região paralela.

- Default(none)

Faz com que seja necessário declarar explicitamente o escopo de cada variável, tornando o código mais claro e ajudando a evitar erros de

escopo, especialmente em programas complexos onde muitas variáveis estão envolvidas.

2.4. Flag

```
gcc -o tarefa6 tarefa6.c -lm -fopenmp  
./tarefa6
```

2.5. Etapas Para Resolução Da Tarefa

1. Implementar a versão sequencial do Monte Carlo para pi;
2. Paralelizar com `#pragma omp parallel for` e mostrar o resultado errado;
3. Explicar a race condition;
4. Corrigir com `#pragma omp critical` e medir o impacto no tempo;
5. Reestruturar com `#pragma omp parallel + #pragma omp for` separados (usando `count_local`);
6. Shared;
7. Private;
8. Firstprivate;
9. Lastprivate;
10. Default(none);

2.6. RESOLUÇÃO EM CÓDIGO DA TAREFA

Segue no apêndice.

2.7. OUTPUT

Segue no apêndice.

3. CONCLUSÃO

A versão inicial com `#pragma omp parallel for` demonstrou o problema da condição de corrida (race condition): o valor de pi obtido (0.765) ficou completamente distorcido, evidenciando que a operação `count++` não é atômica e que múltiplas threads sobrescrevem os incrementos umas das outras quando acessam uma variável compartilhada sem proteção.

A correção com `#pragma omp critical` resolveu o problema de corretude, porém introduziu um problema de desempenho significativo, sendo muito mais lenta que a

versão estruturada. Isso ocorre porque o critical é acionado a cada iteração do loop, forçando as threads a aguardarem em fila para acessar a seção crítica.

A reestruturação com #pragma omp parallel e #pragma omp for separados, utilizando contadores locais (count_local), mostrou-se a abordagem mais eficiente, já que cada thread acumula seus resultados localmente e acessa a seção crítica apenas uma vez ao final, eliminando praticamente toda a contenção.

Os testes com as cláusulas de escopo revelaram que:

- Shared aplicada a variáveis como x e y gera resultados instáveis, pois as threads podem sobrescrever os valores umas das outras entre a geração e a utilização (race condition);
- Private corrige esse problema ao fornecer a cada thread sua própria cópia das variáveis, sem inicialização;
- Firstprivate inicializa cada cópia com o valor original, sendo útil para variáveis como seeds;
- Lastprivate preserva o valor da última iteração lógica após a região paralela, útil quando esse valor é necessário posteriormente;
- Default(none) obriga a declaração explícita de todas as variáveis, funcionando como uma rede de segurança contra race conditions.

4. APÊNDICE

4.1. RESOLUÇÃO

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>
#include <time.h>

#define N 1000000

int meu_rand(unsigned int *seed) { // gerar seed
    *seed = *seed * 1103515245 + 12345;
    return (*seed >> 16) & 0x7fff;
}

// versao com apenas pragma omp parallel for
void versao_parallel_for() {
    //contagem de tempo início
    double inicio = omp_get_wtime();

    srand(time(NULL));
    int count = 0;
```

```

// declarar o quadrado de 0 a 1 usando rand()
#pragma omp parallel for //fazer a paralisação
for(int i=0; i<N; i++) {
    double x = (double)rand()/RAND_MAX;
    double y = (double)rand()/RAND_MAX;

    if (x*x + y*y <= 1) {
        count++;
    }
}
// calculo pro valor de pi
double pi = (4.0 * count)/N;
// tempo final
double fim = omp_get_wtime();

printf("#pragma omp parallel for -> pi: %.6f\n", pi);
printf("Tempo: %fs\n\n", fim - inicio);
}

```

```

// versao com critical e parallel
void versao_parallel_for_mais_critical() {
    //contagem de tempo início
    double inicio = omp_get_wtime();

    srand(time(NULL));
    int count = 0;

    // declarar o quadrado de 0 a 1 usando rand()
    #pragma omp parallel for //fazer a paralisação
    for(int i=0; i<N; i++) {
        double x = (double)rand()/RAND_MAX;
        double y = (double)rand()/RAND_MAX;

        if (x*x + y*y <= 1) {
            #pragma omp critical
            {
                count++;
            }
        }
    }
}

// calculo pro valor de pi
double pi = (4.0 * count)/N;
// tempo final
double fim = omp_get_wtime();

printf("#pragma omp parallel for + critical -> pi: %.6f\n", pi);

```

```

    printf("Tempo: %fs\n\n", fim - inicio);
}

// versao com parallel + omp for
void versao_parallel_mais_for() {
    //contagem de tempo início
    double inicio = omp_get_wtime();

    srand(time(NULL));
    int count = 0;

    // declarar o quadrado de 0 a 1 usando rand()
    #pragma omp parallel //só abre as threads
    {
        int count_local = 0;

        #pragma omp for // só divide o loop
        for(int i=0; i<N; i++) {
            double x = (double)rand()/RAND_MAX;
            double y = (double)rand()/RAND_MAX;

            if (x*x + y*y <= 1) {
                count_local++;
            }
        }
        #pragma omp critical
        {
            count += count_local;
        }
    }

    // calculo pro valor de pi
    double pi = (4.0 * count)/N;
    // tempo final
    double fim = omp_get_wtime();

    printf("#pragma omp parallel + for -> pi: %.6f\n", pi);
    printf("Tempo: %fs\n\n", fim - inicio);
}

// versao com parallel e clausula shared
void versao_parallel_mais_shared() {
    //contagem de tempo início
    double inicio = omp_get_wtime();

    srand(time(NULL));
    double x,y;

```

```

int count = 0;

// declarar o quadrado de 0 a 1 usando rand()
#pragma omp parallel shared(x, y, count) // indicar que vai ser
compartilhado essas variáveis
{
    int count_local = 0;

    #pragma omp for // só divide o loop
    for(int i=0; i<N; i++) {
        x = (double)rand()/RAND_MAX;
        y = (double)rand()/RAND_MAX;

        if (x*x + y*y <= 1) {
            count_local++;
        }
    }
    #pragma omp critical
    {
        count += count_local;
    }
}

// calculo pro valor de pi
double pi = (4.0 * count)/N;
// tempo final
double fim = omp_get_wtime();

printf("#pragma omp parallel + shared -> pi: %.6f\n", pi);
printf("Tempo: %fs\n\n", fim - inicio);
}

// versao com parallel e clausula private
void versao_parallel_mais_private() {
    //contagem de tempo início
    double inicio = omp_get_wtime();

    srand(time(NULL));
    double x,y;
    int count = 0;

    // declarar o quadrado de 0 a 1 usando rand()
    #pragma omp parallel private(x, y) // indicar que vai ser privado
    {
        int count_local = 0;

        #pragma omp for // só divide o loop
        for(int i=0; i<N; i++) {

```

```

        x = (double)rand()/RAND_MAX;
        y = (double)rand()/RAND_MAX;

        if (x*x + y*y <= 1) {
            count_local++;
        }
    }
    #pragma omp critical
    {
        count += count_local;
    }
}

// calculo pro valor de pi
double pi = (4.0 * count)/N;
// tempo final
double fim = omp_get_wtime();

printf("#pragma omp parallel + private -> pi: %.6f\n", pi);
printf("Tempo: %fs\n\n", fim - inicio);
}

// versao com parallel e clausula first private
void versao_parallel_mais_firstprivate() {
    //contagem de tempo início
    double inicio = omp_get_wtime();

    srand(time(NULL));
    unsigned int seed = 2;
    int count = 0;

    // declarar o quadrado de 0 a 1 usando rand()
    #pragma omp parallel firstprivate(seed) // indicar o first private vai
    trabalhar usando o seed 2 como base
    {
        int count_local = 0;

        #pragma omp for // só divide o loop
        for(int i=0; i<N; i++) {
            double x = (double)meu_rand(&seed)/0x7fff; //rand_r já que usa a
            seed passada, maior controle
            double y = (double)meu_rand(&seed)/0x7fff;

            if (x*x + y*y <= 1) {
                count_local++;
            }
        }
    }
    #pragma omp critical

```



```

        {
            count += count_local;
        }
    }

    // calculo pro valor de pi
    double pi = (4.0 * count)/N;
    // tempo final
    double fim = omp_get_wtime();

    printf("#pragma omp parallel + Firstprivate -> pi: %.6f\n", pi);
    printf("Tempo: %fs\n\n", fim - inicio);
}

// versao com parallel e clausula last private
void versao_parallel_mais_lastprivate() {
    //contagem de tempo início
    double inicio = omp_get_wtime();
    double x, y; // aqui devem ser declarados fora da região paralela

    srand(time(NULL));
    int count = 0;

    // declarar o quadrado de 0 a 1 usando rand()
    #pragma omp parallel
    {
        int count_local = 0;

        #pragma omp for lastprivate(x,y) //já que quem controla das iterações é
o for
        for(int i=0; i<N; i++) {
            x = (double)rand()/RAND_MAX;
            y = (double)rand()/RAND_MAX;

            if (x*x + y*y <= 1) {
                count_local++;
            }
        }
        #pragma omp critical
        {
            count += count_local;
        }
    }

    // calculo pro valor de pi
    double pi = (4.0 * count)/N;
    // tempo final
    double fim = omp_get_wtime();

```

```

printf("#pragma omp parallel + Lastprivate -> pi: %.6f\n", pi);
printf("Último x: %f, Último y: %f\n", x, y);
printf("Tempo: %fs\n\n", fim - inicio);
}

// versao com parallel e clausula default
void versao_parallel_mais_default() { // versão estruturada com tudo
    explícito
    //contagem de tempo início
    double inicio = omp_get_wtime();
    double x, y; // aqui devem ser declarados fora da região paralela

    srand(time(NULL));
    int count = 0;

    // declarar o quadrado de 0 a 1 usando rand()
    #pragma omp parallel default(none) shared(count) private(x,y)
    {
        int count_local = 0;

        #pragma omp for
        for(int i=0; i<N; i++) {
            x = (double)rand()/RAND_MAX;
            y = (double)rand()/RAND_MAX;

            if (x*x + y*y <= 1) {
                count_local++;
            }
        }
        #pragma omp critical
        {
            count += count_local;
        }
    }

    // calculo pro valor de pi
    double pi = (4.0 * count)/N;
    // tempo final
    double fim = omp_get_wtime();

    printf("#pragma omp parallel + default -> pi: %.6f\n", pi);
    printf("Tempo: %fs\n\n", fim - inicio);
}

int main() {

```

```

    versao_parallel_for();
    versao_parallel_for_mais_critical();
    versao_parallel_mais_for();
    versao_parallel_mais_shared();
    versao_parallel_mais_private();
    versao_parallel_mais_firstprivate();
    versao_parallel_mais_lastprivate();
    versao_parallel_mais_default();

    return 0;
}

```

4.2. OUTPUT

```

PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa6> ./tarefa6
#pragma omp parallel for -> pi: 0.765448
Tempo: 0.038000s

#pragma omp parallel for + critical -> pi: 3.135696
Tempo: 0.068000s

#pragma omp parallel + for -> pi: 3.142416
Tempo: 0.002000s

#pragma omp parallel + shared -> pi: 3.148896
Tempo: 0.010000s

#pragma omp parallel + private -> pi: 3.145056
Tempo: 0.003000s

#pragma omp parallel + Firstprivate -> pi: 3.146304
Tempo: 0.001000s

#pragma omp parallel + Lastprivate -> pi: 3.145056
|Último x: 0.625507, |Último y: 0.136906
Tempo: 0.002000s

#pragma omp parallel + default -> pi: 3.138336
Tempo: 0.003000s

```